Ranges

Ranges occur everywhere: January to December, 0 to 9, rare to well done, lines 50 through 67, and so on. If Ruby is to help us model reality, it seems natural for it to support these ranges. In fact, Ruby goes one better: it uses ranges to implement sequences and intervals.

Ranges as Sequences

The first and perhaps most natural use of ranges is to express a sequence of values. Sequences have a start point, an end point, and a way to produce successive values in the sequence. In Ruby, these sequences are created using the .. and ... range operators. The two-dot form creates an inclusive range, and the three-dot form creates a range that excludes the specified high value:

```
1..10
"a".."z"
0...3
```

If you are looking for a way to remember which is which, you can imagine the third dot as replacing the high end value. The two-dot form is more common in actual code, and we recommend not switching between the two—it's a subtle distinction and hard to read.

You can convert a range to an array using the to_a method and convert it to an Enumerator using to_enum.

```
(1..10).to_a       # => [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
('bar'..'bat').to_a # => ["bar", "bas", "bat"]
enum = ('bar'..'bat').to_enum
enum.next          # => "bar"
enum.next          # => "bas"
```

Sometimes people worry that ranges take a lot of memory. That's not an issue: the range 1..100000 is held as a Range object containing references to two Fixnum objects. However, convert a range into an array, and all that memory will get used.

Ruby also allows you to specify ranges that have no beginning or no end. While this can be useful for generating infinite sequences, it's also pretty useful to define a subrange for an array or string. A range starting with [..x] goes from the beginning of the sequence to index x, while [x..] goes from x to the end of the sequence:

```
arr = [1, 2, 3, 4, 5, 6]
arr[..2]  # => [1, 2, 3]
arr[2..]  # => [3, 4, 5, 6]
```

Ranges have methods that let you iterate over them and test their contents in a variety of ways:

```
digits = 0..9
digits.include?(5)         # => true
digits.max                 # => 9
digits.reject {|i| i < 5 } # => [5, 6, 7, 8, 9]
digits.reduce(:+)          # => 45
```

So far we've shown ranges of numbers and strings. However, as you'd expect from an object-oriented language, Ruby ranges can be based on objects that you define. The only constraints